WALKTHROUGH LAB — WEEK 6

C Standard Library: Hands-On Lab

Estimated time: 2–3 hours

Prerequisites: gcc installed, Linux/macOS terminal or WSL on Windows

---

Setup

Create a directory for today's lab:

```
mkdir week6_lab
cd week6_lab
```

You will create one `.c` file per exercise. Compile each with:

```
gcc -Wall -o output_name source_file.c
```

PART 3: Strings

Exercise 3.1 — String Basics

Step 1: Create `ex8_strings.c`:

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    // Different ways to declare strings
    char s1[] = "hello";
    char s2[20] = "world";
    char s3[] = {'C', ' ', 'i', 's', ' ', 'f', 'u', 'n', '\0'};

    printf("s1 = \"%s\", length = %zu\n", s1, strlen(s1));
    printf("s2 = \"%s\", length = %zu\n", s2, strlen(s2));
    printf("s3 = \"%s\", length = %zu\n", s3, strlen(s3));

    // sizeof vs strlen
    printf("\nsizeof(s1) = %zu  (array size including \\0)\n", sizeof(s1));
    printf("strlen(s1) = %zu  (string length excluding \\0)\n", strlen(s1));

    // Print each character with its index
    printf("\nCharacters of s1:\n");
    for (int i = 0; i <= (int)strlen(s1); i++) {
        printf("  s1[%d] = '%c' (ASCII %d)%s\n",
               i, s1[i] ? s1[i] : ' ', (unsigned char)s1[i],
               s1[i] == '\0' ? " <- null terminator" : "");
    }
```

```
    return 0;
}
```

Step 2: Compile and run. Observe the null terminator at the end.

Exercise 3.2 — Implementing strlen

Step 1: Create `ex9_mystrlen.c`:

```
#include <stdio.h>
#include <string.h>

// Implement your own strlen
int my_strlen(const char *s) {
    int i = 0;
    while (s[i] != '\0') i++;
    return i;
}

// Alternative: pointer arithmetic
int my_strlen2(const char *s) {
    const char *p = s;
    while (*p != '\0') p++;
    return (int)(p - s);
}
```

```
int main(void) {
    const char *test_cases[] = {"", "a", "hello", "hello world", NULL};

    for (int i = 0; test_cases[i] != NULL; i++) {
        int lib    = (int)strlen(test_cases[i]);
        int mine   = my_strlen(test_cases[i]);
        int mine2  = my_strlen2(test_cases[i]);

        printf("String: \"%-12s\"  strlen=%d  my_strlen=%d  my_strlen2=%d  %s\n",
               test_cases[i], lib, mine, mine2,
               (lib==mine && mine==mine2) ? "OK" : "MISMATCH!");
    }
    return 0;
}
```

Step 2: Compile and run. All should show “OK”.

Exercise 3.3 — strcpy, strcat, strcmp

Step 1: Create `ex10_strops.c`:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
```

```c
int main(void) {
    // ---- strcpy ----
    char dest[50];
    strcpy(dest, "Hello");
    printf("After strcpy: \"%s\"\n", dest);

    // Safe version with strncpy
    char safe[10];
    strncpy(safe, "This is too long for the buffer!", sizeof(safe)-1);
    safe[sizeof(safe)-1] = '\0';  // always null-terminate
    printf("After strncpy (safe): \"%s\"\n", safe);

    // ---- strcat ----
    char result[50] = "Hello";
    strcat(result, ", ");
    strcat(result, "World");
    strcat(result, "!");
    printf("After strcat: \"%s\"\n", result);

    // ---- strcmp ----
    const char *words[] = {"banana", "apple", "cherry", "apple", NULL};
    printf("\nComparisons:\n");
    for (int i = 0; words[i]; i++) {
        for (int j = i+1; words[j]; j++) {
            int cmp = strcmp(words[i], words[j]);
```

```
            printf("  strcmp(\"%s\", \"%s\") = %d  (%s)\n",
                   words[i], words[j], cmp,
                   cmp < 0 ? "first is less" :
                   cmp > 0 ? "first is greater" : "equal");
        }
    }

    // ---- Dynamic string duplication ----
    const char *original = "Dynamically allocated copy";
    char *copy = malloc(strlen(original) + 1);
    if (!copy) { perror("malloc"); return 1; }
    strcpy(copy, original);
    printf("\nOriginal: %s\n", original);
    printf("Copy:     %s\n", copy);
    printf("Same address? %s\n", (original == copy) ? "yes" : "no");
    printf("Same content? %s\n", strcmp(original, copy)==0 ? "yes" : "no");
    free(copy);

    return 0;
}
```

Step 2: Compile with `-Wall` and run.

Exercise 3.4 — sprintf and sscanf

Step 1: Create `ex11_sprintf.c`:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
    // ---- sprintf: build a filename ----
    char filename[100];
    for (int run = 1; run <= 3; run++) {
        sprintf(filename, "output_run%02d.txt", run);
        printf("Filename for run %d: %s\n", run, filename);
    }

    // ---- sprintf: build a formatted message ----
    char msg[200];
    int pos = 0;
    pos += sprintf(msg + pos, "[");
    const char *items[] = {"alpha", "beta", "gamma", NULL};
    for (int i = 0; items[i]; i++) {
        if (i > 0) pos += sprintf(msg + pos, ", ");
        pos += sprintf(msg + pos, "\"%s\"", items[i]);
    }
    pos += sprintf(msg + pos, "]");
    printf("\nJSON array: %s\n", msg);
```

```
    // ---- sscanf: parse a CSV line ----
    char line[] = "2024,03,15,14,30,00";
    int year, month, day, hour, minute, second;
    int parsed = sscanf(line, "%d,%d,%d,%d,%d,%d",
                        &year, &month, &day, &hour, &minute, &second);
    printf("\nParsed %d fields from \"%s\":\n", parsed, line);
    printf("  Date: %04d-%02d-%02d\n", year, month, day);
    printf("  Time: %02d:%02d:%02d\n", hour, minute, second);

    // ---- sscanf: parse key=value ----
    char pair[] = "temperature=23.5";
    char key[50]; float value;
    sscanf(pair, "%49[^=]=%f", key, &value);
    printf("\nKey: \"%s\", Value: %.1f\n", key, value);

    return 0;
}
```

Step 2: Compile and run. Observe the incremental string building with `msg + pos`.

Exercise 3.5 — String Search with strchr/strstr

Step 1: Create `ex12_search.c`:

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char sentence[] = "The quick brown fox jumps over the lazy dog";

    // strchr: find a character
    char *found = strchr(sentence, 'o');
    while (found != NULL) {
        printf("Found 'o' at index %ld: ...%.*s...\n",
               found - sentence,
               7, found > sentence+3 ? found-3 : sentence);
        found = strchr(found + 1, 'o');  // find next occurrence
    }

    // strstr: find a substring
    printf("\nSearching for \"fox\":\n");
    char *pos = strstr(sentence, "fox");
    if (pos) {
        printf("  Found at index %ld\n", pos - sentence);
        printf("  From there: \"%s\"\n", pos);
    }

    // strtok: split by spaces
    printf("\nWords in sentence:\n");
    char copy[100];
```

```
    strncpy(copy, sentence, sizeof(copy)-1);
    copy[sizeof(copy)-1] = '\0';

    char *token = strtok(copy, " ");
    int wordnum = 0;
    while (token != NULL) {
        printf("  Word %2d: %s\n", wordnum++, token);
        token = strtok(NULL, " ");
    }

    return 0;
}
```

Step 2: Compile and run. Note that `strtok` modifies the string (check `copy` after tokenising).

PART 4: Memory Management

Exercise 4.1 — malloc, calloc, realloc, free

Step 1: Create `ex13_memory.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```c
int main(void) {
    // ---- malloc ----
    int *arr = malloc(5 * sizeof(int));
    if (!arr) { perror("malloc"); return 1; }

    // Uninitialised: may contain garbage
    printf("malloc (uninitialised): ");
    for (int i = 0; i < 5; i++) printf("%d ", arr[i]);
    printf("\n");

    // Fill it
    for (int i = 0; i < 5; i++) arr[i] = (i+1) * 10;
    printf("After filling:          ");
    for (int i = 0; i < 5; i++) printf("%d ", arr[i]);
    printf("\n");

    // ---- realloc: grow the array ----
    int *bigger = realloc(arr, 10 * sizeof(int));
    if (!bigger) { perror("realloc"); free(arr); return 1; }
    arr = bigger;  // arr now points to the new (possibly moved) memory

    for (int i = 5; i < 10; i++) arr[i] = (i+1) * 10;
    printf("After realloc to 10:    ");
    for (int i = 0; i < 10; i++) printf("%d ", arr[i]);
    printf("\n");
```

```
    free(arr);
    arr = NULL;  // good practice: null the pointer after freeing

    // ---- calloc ----
    int *zeroed = calloc(8, sizeof(int));
    if (!zeroed) { perror("calloc"); return 1; }
    printf("\ncalloc (zero-initialised): ");
    for (int i = 0; i < 8; i++) printf("%d ", zeroed[i]);
    printf("\n");
    free(zeroed);

    // ---- memset ----
    char buffer[20];
    memset(buffer, 0, sizeof(buffer));
    printf("\nAfter memset to 0: all bytes are %d\n", buffer[0]);
    memset(buffer, 'X', 10);
    buffer[10] = '\0';
    printf("After memset first 10 to 'X': \"%s\"\n", buffer);

    return 0;
}
```

Step 2: Compile with `gcc -Wall -o ex13 ex13_memory.c` and run.

Step 3: Use valgrind to check for memory leaks:

```
valgrind --leak-check=full ./ex13
```

All memory should be freed with no errors.

Exercise 4.2 — Dynamic String Array

Step 1: Create `ex14_dynstrings.c`:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char *argv[]) {
    // Read N strings from the user, store dynamically
    int n;
    printf("How many strings? ");
    scanf("%d", &n);

    // Consume newline
    int ch;
    while ((ch = getchar()) != '\n' && ch != EOF);

    // Array of char pointers (each will point to a malloced string)
    char **strings = malloc(n * sizeof(char *));
    if (!strings) { perror("malloc strings"); return 1; }
```

```
    char buf[256];
    for (int i = 0; i < n; i++) {
        printf("String %d: ", i+1);
        if (fgets(buf, sizeof(buf), stdin) == NULL) break;

        // Remove trailing newline
        int len = strlen(buf);
        if (len > 0 && buf[len-1] == '\n') buf[--len] = '\0';

        // Allocate exactly the right amount and copy
        strings[i] = malloc(len + 1);
        if (!strings[i]) { perror("malloc string"); return 1; }
        strcpy(strings[i], buf);
    }

    // Print all strings
    printf("\nYou entered:\n");
    for (int i = 0; i < n; i++) {
        printf("  [%d] \"%s\" (length %zu)\n", i, strings[i], strlen(strings[i]));
    }

    // Sort them using strcmp
    // Simple bubble sort
    for (int i = 0; i < n-1; i++) {
        for (int j = 0; j < n-i-1; j++) {
```

```
            if (strcmp(strings[j], strings[j+1]) > 0) {
                char *tmp = strings[j];
                strings[j] = strings[j+1];
                strings[j+1] = tmp;
            }
        }
    }

    printf("\nSorted:\n");
    for (int i = 0; i < n; i++) {
        printf("  [%d] \"%s\"\n", i, strings[i]);
    }

    // Free everything
    for (int i = 0; i < n; i++) {
        free(strings[i]);
        strings[i] = NULL;
    }
    free(strings);

    return 0;
}
```

Step 2: Compile and run. Enter several strings and observe them sorted.

PART 5: Random Numbers and Time

Exercise 5.1 — Random and Time

Step 1: Create `ex15_random.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void) {
    // Show that without seeding, sequence is the same every run
    printf("Without seeding:\n  ");
    for (int i = 0; i < 5; i++) printf("%ld ", random() % 100);
    printf("\n");

    // Now seed with time
    unsigned int seed = (unsigned int)time(NULL);
    srandom(seed);
    printf("With seed %u:\n  ", seed);
    for (int i = 0; i < 5; i++) printf("%ld ", random() % 100);
    printf("\n");

    // Roll a dice 20 times
    printf("\n20 dice rolls (1-6):\n  ");
    for (int i = 0; i < 20; i++) {
```

```
        printf("%ld ", (random() % 6) + 1);
    }
    printf("\n");

    // Simple histogram
    int freq[7] = {0};
    int rolls = 60000;
    for (int i = 0; i < rolls; i++) {
        freq[(random() % 6) + 1]++;
    }
    printf("\nHistogram of %d dice rolls:\n", rolls);
    for (int face = 1; face <= 6; face++) {
        printf("  %d: %5d (%.1f%%)\n", face, freq[face], 100.0*freq[face]/rolls);
    }

    // Time measurement
    time_t start = time(NULL);
    long sum = 0;
    for (long i = 0; i < 100000000L; i++) sum += i;
    time_t end = time(NULL);
    printf("\nSum to 100M = %ld, took ~%ld second(s)\n", sum, end-start);

    return 0;
}
```

Step 2: Compile and run twice. Observe different sequences due to time-based seeding.

PART 6: Command Line Arguments

Exercise 6.1 — Basic Arguments

Step 1: Create `ex16_args.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void print_usage(const char *program_name) {
    printf("Usage: %s <name> <age> [greeting]\n", program_name);
    printf("  name:     your name\n");
    printf("  age:      your age (integer)\n");
    printf("  greeting: optional greeting word (default: Hello)\n");
    printf("Example: %s Alice 21 Hi\n", program_name);
}

int main(int argc, char *argv[]) {
    // Print all arguments
    printf("argc = %d\n", argc);
    for (int i = 0; i < argc; i++) {
        printf("argv[%d] = \"%s\"\n", i, argv[i]);
    }
    printf("\n");
```

```c
    // Check minimum arguments
    if (argc < 3) {
        printf("Error: not enough arguments\n");
        print_usage(argv[0]);
        return 1;
    }

    // Parse arguments
    const char *name     = argv[1];
    int         age      = atoi(argv[2]);  // string to int
    const char *greeting = (argc >= 4) ? argv[3] : "Hello";

    // Validate age
    if (age <= 0 || age > 150) {
        printf("Error: age must be between 1 and 150\n");
        return 1;
    }

    printf("%s, %s! You are %d years old.\n", greeting, name, age);

    // Check if name is "admin" (case-sensitive)
    if (strcmp(name, "admin") == 0) {
        printf("Welcome, administrator!\n");
    }
```

```
    return 0;
}
```

Step 2: Compile and test various invocations:

```
gcc -Wall -o ex16 ex16_args.c
./ex16                        # too few args
./ex16 Alice 21               # minimum args
./ex16 Alice 21 Hi            # with optional greeting
./ex16 admin 99 Welcome       # special name
./ex16 Bob abc                # invalid age
```

Exercise 6.2 — A Complete Mini-Application

Step 1: Create `ex17_wordcount.c` — a simplified `wc` command:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

typedef struct {
    long lines;
    long words;
    long chars;
```

```
} Stats;

Stats count_file(FILE *f) {
    Stats s = {0, 0, 0};
    int c;
    int in_word = 0;

    while ((c = fgetc(f)) != EOF) {
        s.chars++;
        if (c == '\n') s.lines++;
        if (isspace(c)) {
            in_word = 0;
        } else if (!in_word) {
            in_word = 1;
            s.words++;
        }
    }
    return s;
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s <file1> [file2] ...\n", argv[0]);
        printf("Counts lines, words, and characters in files\n");
        return 1;
    }
```

```
    Stats total = {0, 0, 0};
    int files_ok = 0;

    for (int i = 1; i < argc; i++) {
        FILE *f = fopen(argv[i], "r");
        if (!f) {
            fprintf(stderr, "%s: %s: No such file\n", argv[0], argv[i]);
            continue;
        }

        Stats s = count_file(f);
        printf("%6ld %6ld %6ld  %s\n", s.lines, s.words, s.chars, argv[i]);

        total.lines += s.lines;
        total.words += s.words;
        total.chars += s.chars;
        files_ok++;
        fclose(f);
    }

    if (files_ok > 1) {
        printf("%6ld %6ld %6ld  total\n", total.lines, total.words, total.chars);
    }
```

```
    return (files_ok == 0) ? 1 : 0;
}
```

Step 2: Compile and test:

```
gcc -Wall -o ex17 ex17_wordcount.c
./ex17 students.txt
./ex17 students.txt ex17_wordcount.c
./ex17 nonexistent.txt
```

Step 3: Compare with the real `wc` command:

```
wc students.txt
./ex17 students.txt
```

Are the results the same?

PART 7: Putting It All Together

Final Exercise — Student Database

Step 1: Create `ex18_database.c` — a small student database using all concepts:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define MAX_NAME 30
#define DB_FILE  "database.bin"

typedef struct {
    int   id;
    char  name[MAX_NAME];
    int   year;
    float gpa;
} Student;

void add_student(int id, const char *name, int year, float gpa) {
    FILE *f = fopen(DB_FILE, "ab");
    if (!f) { perror("add_student: fopen"); return; }

    Student s;
    s.id   = id;
    s.year = year;
    s.gpa  = gpa;
    strncpy(s.name, name, MAX_NAME-1);
    s.name[MAX_NAME-1] = '\0';
```

```
    fwrite(&s, sizeof(Student), 1, f);
    fclose(f);
    printf("Added: ID=%d  Name=%s\n", id, name);
}

void list_students(void) {
    FILE *f = fopen(DB_FILE, "rb");
    if (!f) { printf("No database found. Add students first.\n"); return; }

    printf("\n%-5s %-25s %-5s %-5s\n", "ID", "Name", "Year", "GPA");
    printf("%-5s %-25s %-5s %-5s\n", "---", "----", "----", "---");

    Student s;
    int count = 0;
    while (fread(&s, sizeof(Student), 1, f) == 1) {
        printf("%-5d %-25s %-5d %.2f\n", s.id, s.name, s.year, s.gpa);
        count++;
    }
    printf("\nTotal: %d student(s)\n", count);
    fclose(f);
}

void search_student(const char *name_query) {
    FILE *f = fopen(DB_FILE, "rb");
    if (!f) { printf("No database.\n"); return; }
```

```
    Student s;
    int found = 0;
    while (fread(&s, sizeof(Student), 1, f) == 1) {
        if (strstr(s.name, name_query) != NULL) {
            printf("Found: ID=%d  %s  Year=%d  GPA=%.2f\n",
                   s.id, s.name, s.year, s.gpa);
            found++;
        }
    }
    if (!found) printf("No students matching \"%s\"\n", name_query);
    fclose(f);
}

void export_csv(const char *csv_file) {
    FILE *in  = fopen(DB_FILE, "rb");
    FILE *out = fopen(csv_file, "w");
    if (!in || !out) { perror("export"); return; }

    fprintf(out, "ID,Name,Year,GPA\n");
    Student s;
    int count = 0;
    while (fread(&s, sizeof(Student), 1, in) == 1) {
        fprintf(out, "%d,%s,%d,%.2f\n", s.id, s.name, s.year, s.gpa);
        count++;
    }
    printf("Exported %d records to %s\n", count, csv_file);
```

```
    fclose(in);
    fclose(out);
}

void print_usage(const char *prog) {
    printf("Usage:\n");
    printf("  %s add <id> <name> <year> <gpa>\n", prog);
    printf("  %s list\n", prog);
    printf("  %s search <name>\n", prog);
    printf("  %s export <csvfile>\n", prog);
    printf("  %s clear\n", prog);
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        print_usage(argv[0]);
        return 1;
    }

    if (strcmp(argv[1], "add") == 0) {
        if (argc < 6) {
            printf("add requires: <id> <name> <year> <gpa>\n");
            return 1;
        }
        int   id   = atoi(argv[2]);
        char *name = argv[3];
```

```
        int   year = atoi(argv[4]);
        float gpa  = (float)atof(argv[5]);
        add_student(id, name, year, gpa);

    } else if (strcmp(argv[1], "list") == 0) {
        list_students();

    } else if (strcmp(argv[1], "search") == 0) {
        if (argc < 3) { printf("search requires a name\n"); return 1; }
        search_student(argv[2]);

    } else if (strcmp(argv[1], "export") == 0) {
        if (argc < 3) { printf("export requires a filename\n"); return 1; }
        export_csv(argv[3] ? argv[3] : "export.csv");

    } else if (strcmp(argv[1], "clear") == 0) {
        remove(DB_FILE);
        printf("Database cleared\n");

    } else {
        printf("Unknown command: %s\n", argv[1]);
        print_usage(argv[0]);
        return 1;
    }
```

```
    return 0;
}
```

Step 2: Compile and run:

```
gcc -Wall -o db ex18_database.c
./db                                # show usage
./db add 1 Alice 3 3.85
./db add 2 Bob 2 3.20
./db add 3 Carol 3 3.95
./db add 4 Dave 1 2.80
./db list
./db search "al"
./db export students_export.csv
cat students_export.csv
xxd database.bin                    # look at binary format
./db clear
./db list                           # should be empty
```

Lab Summary Checklist

By the end of this lab, students should be able to:

- ☐ **Use** `printf` safely with explicit format strings
- ☐ **Read input with** `scanf` and check its return value

- [ ] **Use** `getchar` to process input character by character
- [ ] **Open, read from, write to, and close files with** `fopen` / `fclose`
- [ ] **Write and read binary data with** `fwrite` / `fread`
- [ ] **Write and read formatted text with** `fprintf` / `fscanf`
- [ ] **Use** `fseek` for random file access
- [ ] **Work with C strings:** `strlen`, `strcpy`, `strncpy`, `strcat`, `strcmp`
- [ ] **Build strings incrementally with** `sprintf`
- [ ] **Parse strings with** `sscanf`
- [ ] **Allocate and free memory with** `malloc`, `calloc`, `realloc`, `free`
- [ ] **Use** `memset` and `memcpy` for memory operations
- [ ] **Generate random numbers and seed with** `time()`
- [ ] **Accept and parse command line arguments via** `argc` / `argv`

Common Errors to Watch For

Error	Cause	Fix
Segfault on `fopen` result	Not checking for NULL	Always `if (f == NULL)` check
Garbage output with `%d`	Format string mismatch	Match specifiers to argument types
Infinite loop with `scanf`	Failed conversion leaves data in buffer	Check return value, flush on failure
Off-by-one in string copy	Forgot `+1` for `'\0'`	`malloc(strlen(s)+1)`
strcmp never matches	Used `==` instead of `strcmp()==0`	Use `strcmp` for string comparison
Buffer overflow	`strcpy` into too-small buffer	Use `strncpy` with explicit size limit

Error	Cause	Fix
Memory leak	`malloc` without `free`	Always pair allocations with frees; use valgrind
Dangling pointer	Using ptr after `free`	Set to NULL after freeing
File not flushed	Data in buffer not written	Call `fclose` or `fflush`